SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA14 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation. *(BL3)*

Assessment Tool Weightage: 40%

Duration :60 Mins
Off Class
QUESTIONS: 5 **Total Marks:50**

Annexure A

DESIGN TASK

Code of Conduct:

I V G ABBIRAMY – 192524097 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

V G ABBIRAMY

Annexure A

DESIGN TASK

Experiment 1: Recursive Descent Parser using Python

Aim

To design and implement a **Recursive Descent Parser** in Python for the given grammar, validate the input string, display the parsing steps, identify syntax errors, and display whether the input string is **Accepted** or **Rejected**.

Grammar

$E \rightarrow T\ E'$

$E' \rightarrow +\ T\ E' \mid \varepsilon$

$T \rightarrow F\ T'$

$T' \rightarrow *\ F\ T' \mid \varepsilon$

$F \rightarrow (E) \mid id$

Design

The parser is implemented using the **Recursive Descent Parsing** technique, where each non-terminal in the grammar is represented by a separate function.

- **E()** parses the expression.
- **E1()** parses the remaining part of the expression (+ T E' or ε).
- **T()** parses the term.
- **T1()** parses the remaining part of the term (* F T' or ε).
- **F()** parses a factor (id or (E)).
- **match()** verifies whether the expected token is present.
- **error()** reports syntax errors and rejects the string.

The parser follows the grammar rules recursively until the entire input string is parsed.

Procedure

1. Read the input expression from the user.
2. Split the input into individual tokens.
3. Start parsing from the start symbol **E**.
4. Call the corresponding recursive functions for each grammar rule.
5. Match terminals (id, +, *, (,)).
6. Display each production rule used during parsing.
7. If the complete input is parsed successfully, display **String Accepted**.

8. If an unexpected token is encountered, display a syntax error and **String Rejected**.

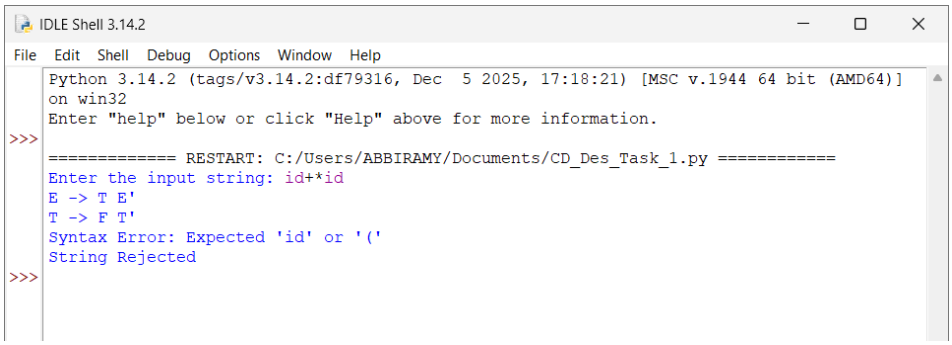
Python Code

```
# Recursive Descent Parser


class Parser:
    def __init__(self, input_string):
        self.tokens = input_string.split()
        self.index = 0

    def current_token(self):
        if self.index < len(self.tokens):
            return self.tokens[self.index]
        return None

    def match(self, expected):
        if self.current_token() == expected:
            print(f"Matched: {expected}")
            self.index += 1
        else:
            self.error(f"Expected '{expected}'")

    def error(self, message):
        raise Exception(f"Syntax Error: {message}")

    def E(self):
        print("E -> T E'")
        self.T()
        self.E1()

    def E1(self):
        if self.current_token() == "+":
            print("E' -> + T E'")
            self.match("+")
```

```
        self.T()
        self.E1()
    else:
        print("E' -> ε")

def T(self):
    print("T -> F T'")
    self.F()
    self.T1()

def T1(self):
    if self.current_token() == "*":
        print("T' -> * F T'")
        self.match("*")
        self.F()
        self.T1()
    else:
        print("T' -> ε")

def F(self):
    token = self.current_token()

    if token == "id":
        print("F -> id")
        self.match("id")

    elif token == "(":
        print("F -> (E)")
        self.match("(")
        self.E()
        self.match(")")

    else:
```

```
            self.error("Expected 'id' or '('")

    def parse(self):
        try:
            self.E()

            if self.current_token() is None:
                print("\nString Accepted")
            else:
                print("\nString Rejected")
        except Exception as e:
            print(e)
            print("String Rejected")


# Main Program
input_string = input("Enter the input string: ")

parser = Parser(input_string)
parser.parse()
```

Sample Output



Result

A Recursive Descent Parser was successfully designed and implemented in Python for the given grammar. The parser validates the input string, displays the parsing steps, detects syntax errors, and correctly reports whether the given input string is Accepted or Rejected.

Experiment 2: Design an LL(1) Parser using Lex and Yacc

Aim

To design and implement an **LL(1) Parser** using **Lex and Yacc** for the given grammar, display the parsing steps, validate the input string, handle syntax errors, and display whether the input string is **Accepted** or **Rejected**.

Grammar

$E \rightarrow T\ E'$

$E' \rightarrow +\ T\ E' \mid \varepsilon$

$T \rightarrow F\ T'$

$T' \rightarrow *\ F\ T' \mid \varepsilon$

$F \rightarrow (E) \mid id$

Design

The program consists of two phases:

Lexical Analyzer

- Reads the input expression.
- Splits it into tokens (id, +, *, (,)).
- Displays the list of tokens.

LL(1) Parser

- Parses the tokens according to the grammar.
- Displays each production rule used.
- Validates the input string.
- Detects syntax errors.
- Displays Accepted or Rejected.

Procedure

1. Read the input string.
2. Perform lexical analysis by converting the input into tokens.
3. Display the generated tokens.
4. Start parsing from the start symbol E.
5. Match tokens according to the grammar rules.
6. Display every parsing step.
7. If all tokens are parsed successfully, display String Accepted.
8. Otherwise, display Syntax Error and String Rejected.

Python Code

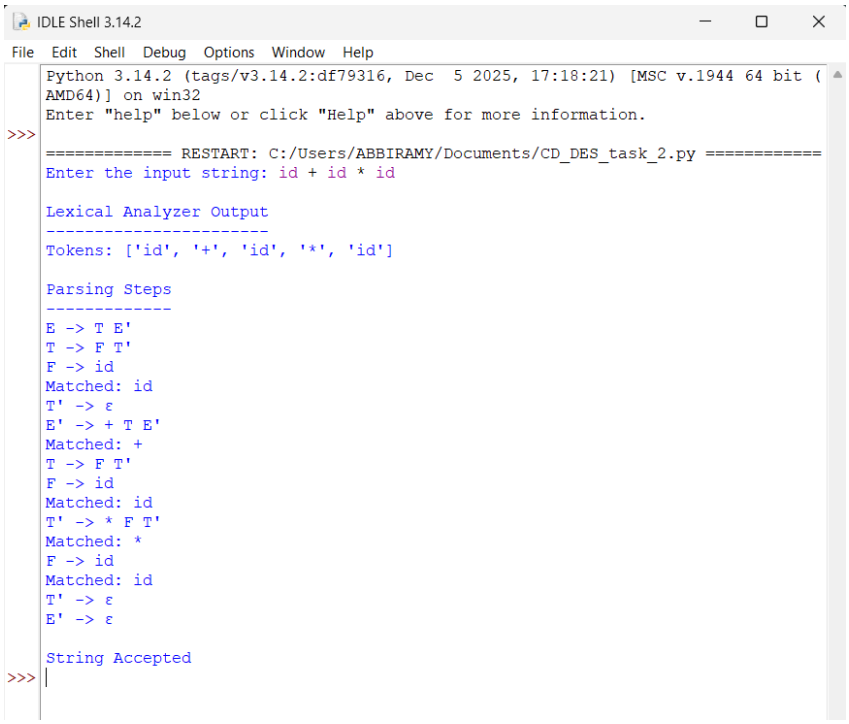
```
# LL(1) Parser with Lexical Analyzer


class Parser:
    def __init__(self, tokens):
        self.tokens = tokens
        self.index = 0

    def current_token(self):
        if self.index < len(self.tokens):
            return self.tokens[self.index]
        return None

    def match(self, expected):
        if self.current_token() == expected:
            print("Matched:", expected)
            self.index += 1
        else:
            self.error(f"Expected '{expected}'")

    def error(self, msg):
        raise Exception(msg)

    def E(self):
        print("E -> T E'")
        self.T()
        self.E1()

    def E1(self):
        if self.current_token() == "+":
            print("E' -> + T E'")
            self.match("+")
            self.T()
```

```
            self.E1()
        else:
            print("E' -> ε")

    def T(self):
        print("T -> F T'")
        self.F()
        self.T1()

    def T1(self):
        if self.current_token() == "*":
            print("T' -> * F T'")
            self.match("*")
            self.F()
            self.T1()
        else:
            print("T' -> ε")

    def F(self):
        if self.current_token() == "id":
            print("F -> id")
            self.match("id")

        elif self.current_token() == "(":
            print("F -> (E)")
            self.match("(")
            self.E()
            self.match(")")

        else:
            self.error("Expected 'id' or '('")

    def parse(self):
```

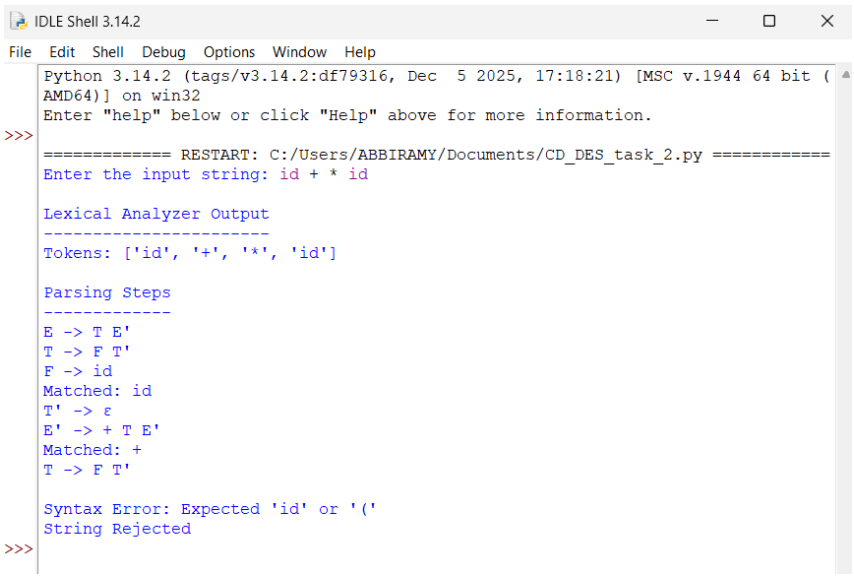
```
        self.E()

        if self.index == len(self.tokens):
            print("\nString Accepted")
        else:
            print("\nString Rejected")


# ---------- Lexical Analyzer ----------

def lexer(expression):
    tokens = expression.split()

    valid = {"id", "+", "*", "(", ")"}

    for token in tokens:
        if token not in valid:
            raise Exception(f"Invalid Token: {token}")

    return tokens


# ---------- Main Program ----------

try:
    expression = input("Enter the input string: ")

    tokens = lexer(expression)

    print("\nLexical Analyzer Output")
    print("-----------------------")
    print("Tokens:", tokens)
```

```
    print("\nParsing Steps")
    print("-------------")

    parser = Parser(tokens)
    parser.parse()

except Exception as e:
    print("\nSyntax Error:", e)
    print("String Rejected")
```

Output



Syntax Error Example



Result

A **Lexical Analyzer** and an **LL(1) Parser** were successfully implemented in Python. The lexical analyzer tokenizes the input string, the parser validates it according to the given grammar, displays the parsing steps, detects syntax errors, and correctly reports whether the input string is **Accepted** or **Rejected**.

Experiment 3: Shift-Reduce Parser using Python

Aim

To design and implement a **Shift-Reduce Parser** in Python for the given grammar, display the **Shift** and **Reduce** operations, validate the input string, handle syntax errors, and display whether the input string is **Accepted** or **Rejected**.

Grammar

$E \rightarrow E + T$

$E \rightarrow T$

$T \rightarrow T * F$

$T \rightarrow F$

$F \rightarrow (E)$

$F \rightarrow id$

Design

The parser is implemented using the **Shift-Reduce Parsing** technique.

- The input string is divided into tokens.
- A **stack** is maintained.
- The parser **shifts** one token at a time from the input to the stack.
- Whenever the top of the stack matches the right-hand side of a grammar production, a **reduce** operation is performed.
- Parsing continues until the stack contains only the start symbol **E** and all input tokens are consumed.
- If parsing succeeds, the string is **Accepted**; otherwise, it is **Rejected**.

Procedure

1. Read the input string from the user.
2. Perform lexical analysis by splitting the input into tokens.
3. Initialize an empty stack.
4. Shift one token at a time from the input to the stack.
5. Check whether the stack matches any grammar production.
6. If a match is found, perform the corresponding Reduce operation.
7. Display every Shift and Reduce operation.
8. Repeat until all input tokens are processed.

9. If the final stack contains only **E**, display **String Accepted**; otherwise display **String Rejected**.

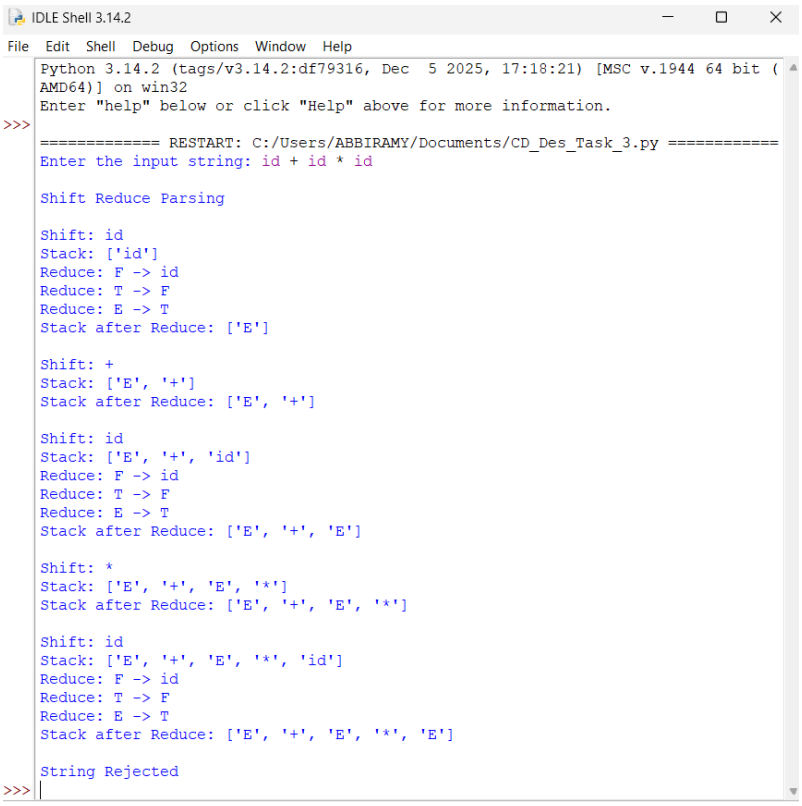
Python Code

```
# Shift Reduce Parser


def reduce_stack(stack):
    changed = True

    while changed:
        changed = False

        # F -> id
        if len(stack) >= 1 and stack[-1] == "id":
            print("Reduce: F -> id")
            stack[-1] = "F"
            changed = True

        # T -> F
        elif len(stack) >= 1 and stack[-1] == "F":
            print("Reduce: T -> F")
            stack[-1] = "T"
            changed = True

        # E -> T
        elif len(stack) >= 1 and stack[-1] == "T":
            print("Reduce: E -> T")
            stack[-1] = "E"
            changed = True

        # T -> T * F
        elif len(stack) >= 3 and stack[-3:] == ["T", "*", "F"]:
            print("Reduce: T -> T * F")
            stack[-3:] = ["T"]
            changed = True
```

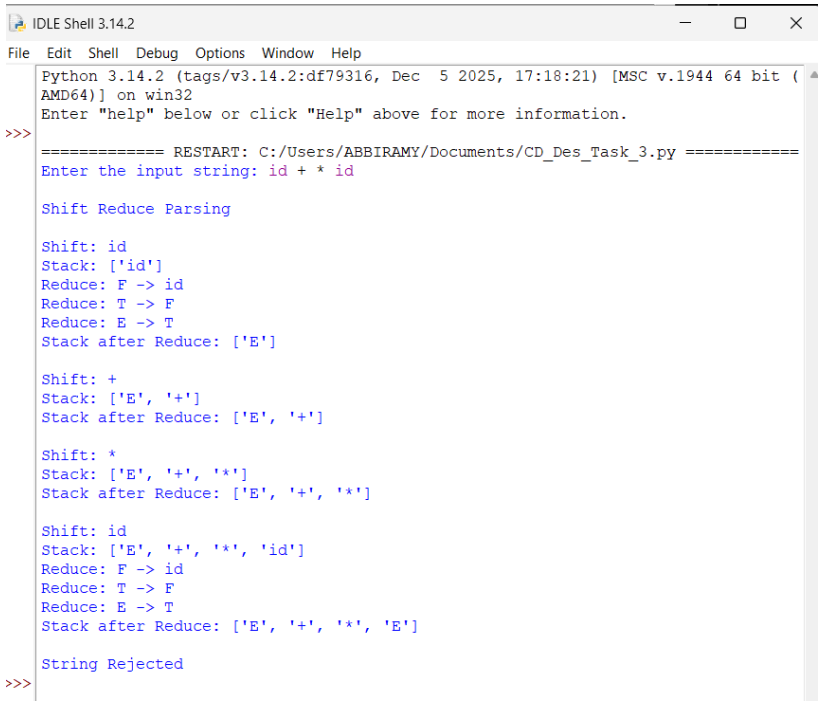
```
        # E -> E + T
        elif len(stack) >= 3 and stack[-3:] == ["E", "+", "T"]:
            print("Reduce: E -> E + T")
            stack[-3:] = ["E"]
            changed = True

        # F -> (E)
        elif len(stack) >= 3 and stack[-3:] == ["(", "E", ")"]:
            print("Reduce: F -> (E)")
            stack[-3:] = ["F"]
            changed = True


def shift_reduce_parser(tokens):
    stack = []

    for token in tokens:
        print("Shift:", token)
        stack.append(token)

        print("Stack:", stack)

        reduce_stack(stack)

        print("Stack after Reduce:", stack)
        print()

    reduce_stack(stack)

    if stack == ["E"]:
        print("String Accepted")
    else:
```

```
        print("String Rejected")


# ---------------- Main Program ----------------

expression = input("Enter the input string: ")

tokens = expression.split()

valid = {"id", "+", "*", "(", ")"}

for token in tokens:
    if token not in valid:
        print("Syntax Error: Invalid Token")
        exit()

print("\nShift Reduce Parsing\n")

shift_reduce_parser(tokens)
```

Output



Syntax Error Example



Result

A Shift-Reduce Parser was successfully implemented in Python. The parser performs Shift and Reduce operations according to the given grammar, validates the input string, detects syntax errors, and correctly reports whether the input string is Accepted or Rejected.

Experiment 4: Operator Precedence Parser using Python

Aim

To design and implement an **Operator Precedence Parser** in Python for the given grammar, apply operator precedence rules, display the parsing steps, validate the input string, handle syntax errors, and display whether the input string is **Accepted** or **Rejected**.

Grammar

E → E + E

E → E * E

E → (E)

E → id

Design

The parser is implemented using the **Operator Precedence Parsing** technique.

- The input expression is tokenized into operators and operands.
- A stack is used to perform **Shift** and **Reduce** operations.
- Operator precedence is applied:
    - * has higher precedence than +.
    - Parentheses () have the highest precedence.

- Whenever a reducible pattern is found, it is reduced according to the grammar.
- If the final stack contains only the start symbol **E**, the input string is accepted.

Procedure

1. Read the arithmetic expression from the user.
2. Split the input into tokens.
3. Initialize an empty stack.
4. Shift one token at a time into the stack.
5. Apply operator precedence rules:
    - id → E
    - (E) → E
    - E * E → E
    - E + E → E
6. Display every Shift and Reduce operation.
7. Continue until all input tokens are processed.
8. If the stack contains only **E**, display **String Accepted**.
9. Otherwise, display **String Rejected**.

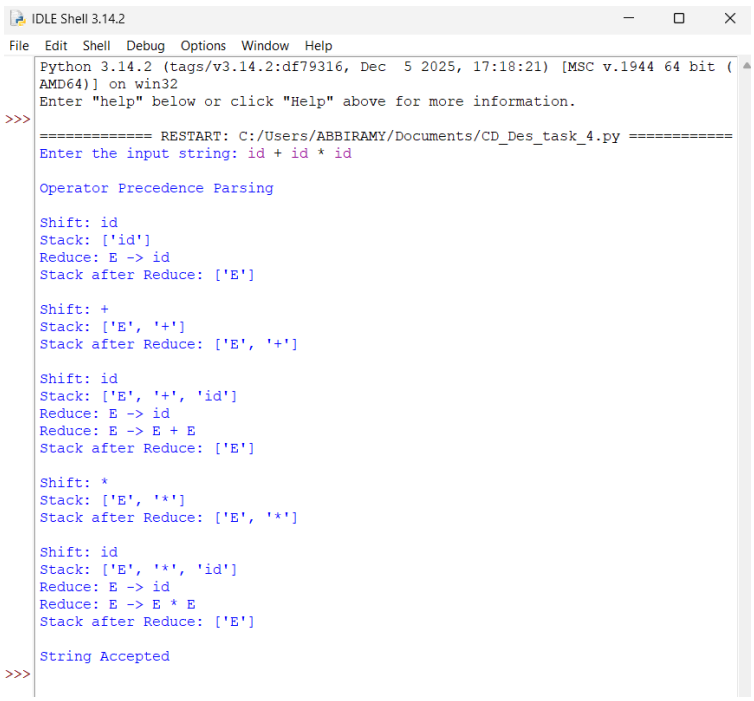
Python Code

```
# Operator Precedence Parser


def reduce_stack(stack):
    changed = True


    while changed:
        changed = False


        # E -> id
        if len(stack) >= 1 and stack[-1] == "id":
            print("Reduce: E -> id")
            stack[-1] = "E"
            changed = True


        # E -> (E)
        elif len(stack) >= 3 and stack[-3:] == ["(", "E", ")"]:
```

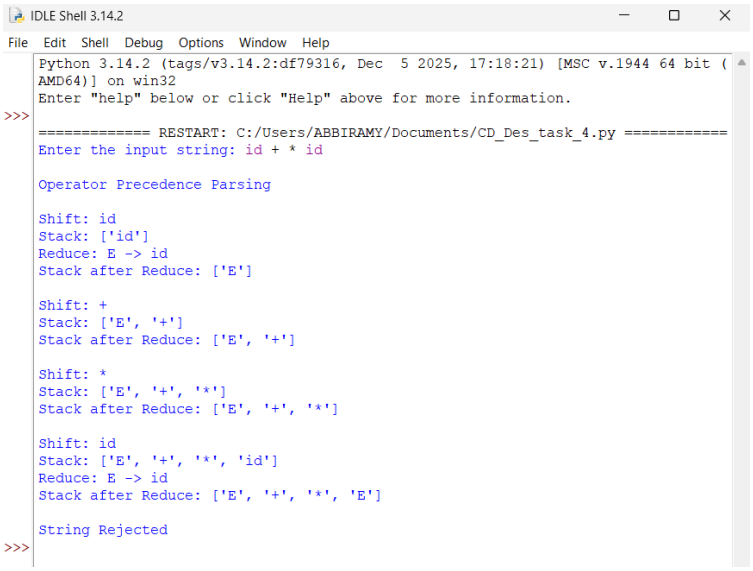
```
            print("Reduce: E -> (E)")
            stack[-3:] = ["E"]
            changed = True

        # E -> E * E
        elif len(stack) >= 3 and stack[-3:] == ["E", "*", "E"]:
            print("Reduce: E -> E * E")
            stack[-3:] = ["E"]
            changed = True

        # E -> E + E
        elif len(stack) >= 3 and stack[-3:] == ["E", "+", "E"]:
            print("Reduce: E -> E + E")
            stack[-3:] = ["E"]
            changed = True


def operator_precedence_parser(tokens):
    stack = []

    valid = {"id", "+", "*", "(", ")"}

    for token in tokens:
        if token not in valid:
            print("Syntax Error: Invalid Token")
            print("String Rejected")
            return

        print("Shift:", token)
        stack.append(token)
        print("Stack:", stack)

        reduce_stack(stack)
```

```
        print("Stack after Reduce:", stack)
        print()

    reduce_stack(stack)

    if stack == ["E"]:
        print("String Accepted")
    else:
        print("String Rejected")


# ---------- Main Program ----------

expression = input("Enter the input string: ")

tokens = expression.split()

print("\nOperator Precedence Parsing\n")

operator_precedence_parser(tokens)
```

Output



Syntax Error Example



Result

An **Operator Precedence Parser** was successfully implemented in Python. The parser applies operator precedence rules (* has higher precedence than +), performs Shift and Reduce operations, displays the parsing steps, detects syntax errors, validates the input string, and correctly reports whether the input string is **Accepted** or **Rejected**.

Experiment 5: LR(0) Parser using Python

Aim

To design and implement a **Lexical Analyzer** and an **LR(0) Parser** in Python for the given grammar, display the parser actions, validate the input string, handle syntax errors, and display whether the input string is **Accepted** or **Rejected**.

Grammar

S → C C

C → c C

C → d

Design

The program consists of two phases:

Lexical Analyzer

- Reads the input string.
- Recognizes the valid symbols **c** and **d**.
- Generates the list of tokens.

LR(0) Parser

- Parses the token sequence according to the grammar.
- Performs **Shift** and **Reduce** operations.
- Displays each parser action.

- Validates the input string.
- Displays **Accepted** or **Rejected**.

Procedure

1. Read the input string.
2. Perform lexical analysis by checking whether every character is either **c** or **d**.
3. Store the tokens.
4. Shift each token onto the stack.
5. Apply the grammar productions whenever possible.
6. Display every **Shift** and **Reduce** action.
7. Continue until the complete input is parsed.
8. If the final stack contains only **S**, display **String Accepted**.
9. Otherwise, display **String Rejected**.

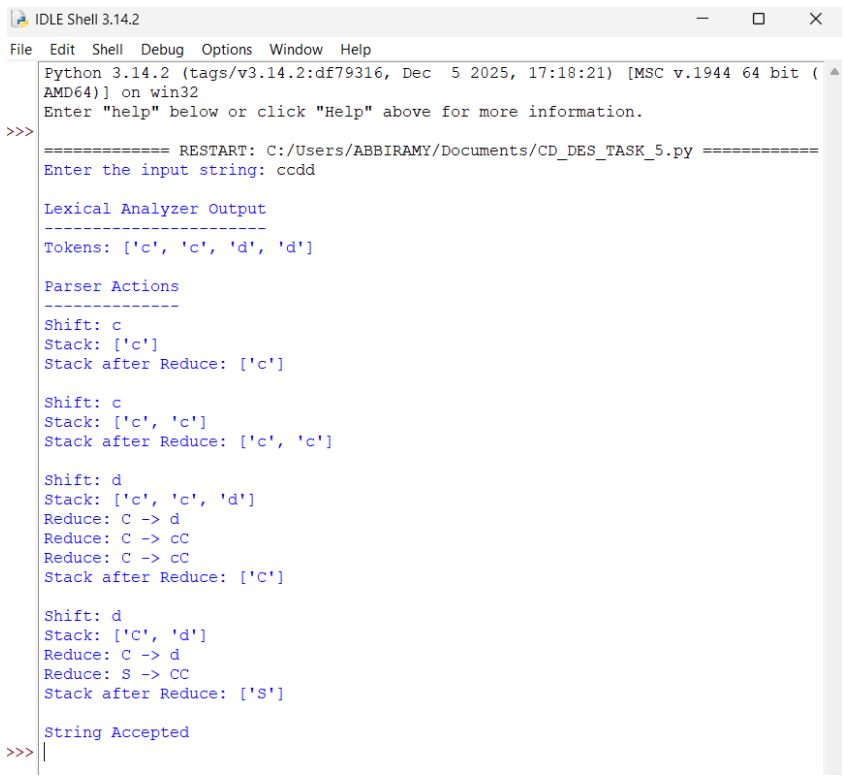
Python Code

```
# LR(0) Parser for
# S -> CC
# C -> cC | d

def reduce_stack(stack):
    changed = True

    while changed:
        changed = False

        # C -> d
        if len(stack) >= 1 and stack[-1] == 'd':
            print("Reduce: C -> d")
            stack[-1] = 'C'
            changed = True

        # C -> cC
        elif len(stack) >= 2 and stack[-2:] == ['c', 'C']:
            print("Reduce: C -> cC")
            stack[-2:] = ['C']
```

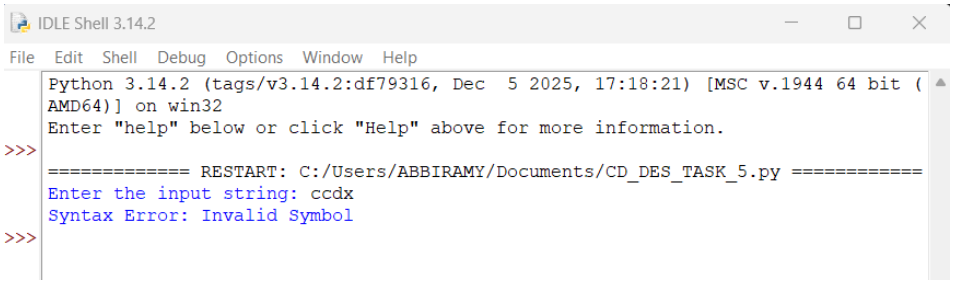
```
            changed = True

        # S -> CC
        elif len(stack) >= 2 and stack[-2:] == ['C', 'C']:
            print("Reduce: S -> CC")
            stack[-2:] = ['S']
            changed = True


def parser(tokens):
    stack = []

    for token in tokens:
        print("Shift:", token)
        stack.append(token)
        print("Stack:", stack)

        reduce_stack(stack)

        print("Stack after Reduce:", stack)
        print()

    reduce_stack(stack)

    if stack == ['S']:
        print("String Accepted")
    else:
        print("String Rejected")


# ---------- Lexical Analyzer ----------

expression = input("Enter the input string: ")
```

```
tokens = list(expression.strip())

valid = {'c', 'd'}

for token in tokens:
    if token not in valid:
        print("Syntax Error: Invalid Symbol")
        exit()

print("\nLexical Analyzer Output")
print("----------------------")
print("Tokens:", tokens)

print("\nParser Actions")
print("--------------")

parser(tokens)
```

Output



Syntax Error Example



Result

A **Lexical Analyzer** and an **LR(0) Parser** were successfully implemented in Python. The lexical analyzer recognizes the valid input symbols, while the parser performs **Shift** and **Reduce** operations according to the given grammar. The parser displays each parser action, validates the input string, detects syntax errors, and correctly reports whether the input string is **Accepted** or **Rejected**.

RUBRICS FOR CALCULATION:

		ng and integrates multiple concepts effectively	integration of most concepts	integration	integration
Design & Implementation	30	Welldesigned solution with systematic and optimal implementa tion	Correct design with minor issues	Basic design with errors or inefficiencies	Incorrect or incomplete design
Parser/Algorithm Construction	20	Accurate constructio n of parser/algo rithm with complete logic	Mostly correct construction with minor errors	Partial construction with missing logic	Incorrect or incomplete construction
Analysis & Validation	15	Insightful analysis with correct validation and justification	Good analysis with reasonable justification	Basic analysis with limited validation	Poor or incorrect analysis

Criteria		**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Understanding of Concepts		25	Demonstrates thorough understandi	Good understandin g with proper	Basic understanding with partial	Poor understanding with no

Documentation & Presentation	10	Wellstructured documentation with diagrams, steps, and clarity	Organized documentation with required details	Basic documentation with missing details	Poor or incomplete documentation